応用情報技術者試験　テクノロジ系　No.2

アルゴリズムとプログラミング

データ構造・アルゴリズム・プログラミング・プログラム言語・その他の言語

目次

1. 2-1 データ構造（12語）
2. 2-2 アルゴリズム（11語）
3. 2-3 プログラミング（5語）
4. 2-4 プログラム言語（5語）
5. 2-5 その他の言語（5語）
6. まとめ

2-1 データ構造 まとめ表

主なデータ構造	
構造	**特徴**
スタック	後入れ先出し（LIFO）。push・pop
キュー	先入れ先出し（FIFO）。enqueue・dequeue
リスト	ポインタで次の要素をつなぐ。挿入・削除が速い
2分探索木	左の子＜親＜右の子。探索は平均 O(log n)
ヒープ	親が子より常に大きい（小さい）完全2分木。優先度付きキューに使う
ハッシュ表	キーから計算した位置に格納。探索は平均 O(1)

2-1 データ構造 データ構造

配列
Array

同じ型のデータを連続した場所に並べたもの

ポイント 添字で直接アクセスできるが、途中への挿入は遅い

スタック
Stack

最後に入れたデータを最初に取り出す構造（LIFO）

ポイント 関数呼出しの戻り先や、逆ポーランド記法の計算に使う

キュー
Queue

最初に入れたデータを最初に取り出す構造（FIFO）

ポイント 待ち行列やバッファに使う

連結リスト
Linked List

各要素が次の要素へのポインタを持つ構造

ポイント 単方向・双方向・循環がある

2分探索木
Binary Search Tree

左の部分木の値＜節の値＜右の部分木の値となる2分木

ポイント 偏ると探索が遅くなるため、AVL木など平衡木を使う

完全2分木
Complete Binary Tree

最下段以外がすべて埋まり、最下段は左から詰まっている2分木

2-1 データ構造 データ構造・木のなぞり方

ヒープ
Heap

親の値が子の値以上（または以下）になる完全2分木

ポイント 最大値（最小値）がすぐ取り出せる

B木
B-tree

1つの節に多くのキーを持つ平衡多分木

ポイント ディスク上のデータベースの索引に使う

ハッシュ表
Hash Table

ハッシュ関数でキーから格納位置を決める表

ポイント 位置がぶつかること（衝突・シノニム）への対策が必要

グラフ
Graph

頂点と辺で、ものどうしのつながりを表す構造

ポイント 隣接行列や隣接リストで表す

深さ優先探索
DFS / Depth-First Search

行けるところまで深く進み、行き止まりで戻る探索

ポイント 前順・間順・後順のなぞり方がある

幅優先探索
BFS / Breadth-First Search

出発点に近い頂点から順に調べる探索

ポイント 最短の手数を求めるのに向く

2-2 アルゴリズム まとめ表

整列の計算量			
整列	平均	最悪	特徴
バブルソート	$O(n^2)$	$O(n^2)$	隣どうしを比べて交換
選択ソート	$O(n^2)$	$O(n^2)$	最小値を選んで先頭へ
挿入ソート	$O(n^2)$	$O(n^2)$	ほぼ整列済みなら速い
クイックソート	$O(n \log n)$	$O(n^2)$	基準値で分けて再帰
マージソート	$O(n \log n)$	$O(n \log n)$	分けて併合。安定
ヒープソート	$O(n \log n)$	$O(n \log n)$	ヒープを使う

2-2 アルゴリズム 探索と整列

線形探索
Linear Search

先頭から順に1つずつ調べる探索

ポイント 計算量 O(n)。番兵を置くと終了判定が減る

2分探索
Binary Search

整列済みのデータを半分に分けながら調べる探索

ポイント 計算量 O(log n)。最大の比較回数は約 log2 n +1

クイックソート
Quick Sort

基準値（ピボット）より小さいグループと大きいグループに分け、再帰的に整列する

ポイント 平均は速いが、最悪 $O(n^2)$

マージソート
Merge Sort

データを半分ずつに分けて整列し、併合（マージ）する

ポイント 常に O(n log n)。安定な整列

安定な整列
Stable Sort

同じ値のデータの並び順が、整列の前後で変わらない整列

計算量
Computational Complexity

アルゴリズムの処理の手間を、データ量nの関数で表したもの

ポイント O記法で表す

2-2 アルゴリズム いろいろなアルゴリズム

再帰
Recursion

関数が自分自身を呼び出すこと

ポイント 終了条件が必要。例: 階乗、ハノイの塔

動的計画法
Dynamic Programming

小さな問題の答えを表に記録し、それを使って大きな問題を解く方法

ポイント 同じ計算を繰り返さない

ダイクストラ法
Dijkstra's Algorithm

重みが0以上のグラフで、最短経路を求める方法

ポイント ネットワークの経路選択（OSPF）にも使う

ユークリッドの互除法
Euclidean Algorithm

2つの数の最大公約数を、割り算の余りを繰り返して求める方法

逆ポーランド記法
後置記法 / Reverse Polish Notation

演算子を後ろに書く記法

例 (A+B)×C は AB+C×。スタックで計算できる

2-3 プログラミング プログラムの性質

再入可能
リエントラント / Reentrant

複数のプロセスから同時に呼ばれても正しく動くプログラム

ポイント データをプロセスごとに分けて持つ

再使用可能
Reusable

一度実行したあと、読み込み直さずに再び実行できるプログラム

再帰的
Recursive

実行中に自分自身を呼び出せるプログラム

再配置可能
Relocatable

主記憶のどこに置いても実行できるプログラム

コーディング規約
Coding Standards

プログラムの書き方をそろえるための決まり

ポイント 読みやすさと保守のしやすさを上げる

2-4 プログラム言語 言語の種類

手続型言語
Procedural Language

処理の手順を順に書いていく言語

例 C、COBOL

オブジェクト指向言語
Object-Oriented Language

データと処理をまとめたオブジェクトを組み合わせて作る言語

例 Java、C++、Python

関数型言語
Functional Language

関数の組合せでプログラムを作る言語

ポイント 副作用を避ける。例: Haskell

論理型言語
Logic Programming Language

事実と規則を書き、問合せに答えを推論させる言語

例 Prolog

スクリプト言語
Script Language

コンパイルせずにインタプリタで手軽に実行できる言語

例 JavaScript、Python

2-5 その他の言語 マークアップとデータ

HTML
えいちてぃーえむえる / HyperText Markup Language

Webページの構造をタグで書く言語

HyperText＝リンクでつながる文書／Markup＝目印をつける／Language＝言語

XML
えっくすえむえる / Extensible Markup Language

利用者が自由にタグを決められるマークアップ言語

ポイント データ交換に使う。DTDやXML Schemaで構造を定める

Extensible＝拡張できる／Markup＝目印付け／Language＝言語

JSON
じぇいそん / JavaScript Object Notation

「名前：値」の組でデータを表す軽い形式

ポイント Web APIのデータのやり取りで多く使う

JavaScript＝言語名／Object＝もの／Notation＝表記法

CSS
しーえすえす / Cascading Style Sheets

Webページの見た目（色・配置など）を指定する言語

Cascading＝段階的に重なる／Style Sheets＝書式の指定

UML
ゆーえむえる / Unified Modeling Language

オブジェクト指向の設計を図で表す標準の書き方

Unified＝統一された／Modeling＝モデルを描く／Language＝言語

まとめ この分野で覚える用語

小分類	用語
2-1 データ構造	配列・スタック・キュー・連結リスト・2分探索木・完全2分木・ヒープ・B木・ハッシュ表・グラフ・深さ優先探索・幅優先探索
2-2 アルゴリズム	線形探索・2分探索・クイックソート・マージソート・安定な整列・計算量・再帰・動的計画法・ダイクストラ法・ユークリッドの互除法・逆ポーランド記法
2-3 プログラミング	再入可能・再使用可能・再帰的・再配置可能・コーディング規約
2-4 プログラム言語	手続型言語・オブジェクト指向言語・関数型言語・論理型言語・スクリプト言語
2-5 その他の言語	HTML・XML・JSON・CSS・UML